

Du code différent en fonction de la plateforme

Table des matières

I. Contexte	3
II. Pourquoi a-t-on besoin de faire varier du code entre les plateformes ?	3
III. Exercice : Appliquez la notion	4
IV. Écrire du code pour une plateforme spécifique	5
V. Exercice : Appliquez la notion	8
VI. Essentiel	9
VII. Auto-évaluation	9
A. Exercice final	9
B. Exercice : Défi.....	11
Solutions des exercices	11

I. Contexte

Durée : 1 h

Environnement de travail : Une application React Native initialisée ou utiliser <https://snack.expo.io/>

Pré-requis : Aucun

Contexte

De par son fonctionnement agnostique de plateforme, React Native permet la plupart du temps de conserver un code généraliste et déclinable quel que soit le média qui utilise l'application. Cependant, il est parfois nécessaire de s'approcher au plus près des comportements ou rendus spécifiques aux plateformes cibles.

Nous allons voir comment React Native permet de décliner du code en fonction d'une plateforme spécifique, permettant ainsi de choisir d'utiliser certains composants ou certaines APIs sur iOS et pas sur Android, ou de faire varier les styles de l'application pour s'approcher au mieux de ce qui se fait en termes de design sur telle ou telle plateforme.

II. Pourquoi a-t-on besoin de faire varier du code entre les plateformes ?

Objectifs

- Comprendre à quel moment on peut avoir à faire varier du code selon la plateforme
- Voir des exemples de variation

Mise en situation

React Native utilise des composants natifs des systèmes d'exploitation au moment du rendu. Quand c'est possible, le moteur va essayer de trouver une équivalence sur les deux plateformes et fournir une abstraction pour que le développeur n'ait pas à coder quelque chose de spécifique à une plateforme.

Malheureusement, Android et iOS étant deux systèmes d'exploitation très différents, il n'est pas toujours possible d'avoir d'équivalence entre l'un et l'autre.

Dans ces conditions, il peut arriver que certaines props ne soient disponibles que sur un système et ignorées sur l'autre. Dans ce cas-là, rien de très grave : cela ne causera pas de plantage, mais dans des cas plus avancés, certains composants ne pourront pas s'afficher du tout sur une plateforme et cela causera un plantage de l'application si on l'utilise sur la mauvaise plateforme.

Par exemple, les composants `TouchableNativeFeedback` et `InputAccessoryView` sont réservés respectivement à Android et iOS :

- `TouchableNativeFeedback` permet le fameux « effet goutte d'eau » lorsque l'on clique sur un bouton, propre à Android et dont les possesseurs de smartphones Android ont bien l'habitude.
- `InputAccessoryView` permet quant à lui de personnaliser certains éléments du clavier système d'Apple, cela n'a donc rien à voir avec Android.

L'utilisation d'ombres sur les éléments

Un autre cas d'utilisation que l'on rencontrera fréquemment est l'utilisation des ombres portées :

- Sur Android, il existe une notion d'élévation, qui permet d'appliquer les effets d'ombres et de relief propre au « Material Design » de Google. Cela prend pour valeur un nombre qui va accentuer l'effet de l'ombre, comme s'il y avait de la hauteur.
- Sur iOS, comme sur le Web, il existe la notion de *shadow box*, où l'on peut préciser un grand nombre de propriétés pour avoir un contrôle optimal sur le design de notre ombre portée, ainsi que sur sa direction.

Et quand bien même c'est l'une des fonctionnalités les plus demandées aujourd'hui sur React Native¹, elle n'existe pas encore.

Il faut donc spécifier à chaque boîte qui le nécessite une propriété `elevation` et des propriétés de `shadow`², ce qui fait d'ailleurs l'objet de tentatives d'abstraction par certains membres de la communauté, par exemple `react-native-shadow-generator`³.

Toutes ces différences ont donc potentiellement des impacts, qu'il convient de prendre en compte. Pour cela, il faut utiliser conditionnellement ces composants.

Nous ne couvrons ici que la partie liée au code JavaScript, mais, plus tard, lorsque l'on enverra notamment des notifications *push*, il sera également nécessaire d'écrire du code lié à une plateforme.

Syntaxe	À retenir
	• Malgré la plus grande tentative d'abstraction entre les plateformes, il existe encore quelques cas où le développeur doit procéder à des différences entre les plateformes. • C'est le cas de composants natifs ou encore de la gestion des ombres dans React Native, mais cela peut aller plus loin, comme des différences fondamentales d'architecture sur des fonctionnalités telles que les notifications <i>push</i>.

Exercice : Appliquez la notion

[solution n°1 p.13]

Exercice

React Native assure...

- ☐ Une compatibilité et une abstraction intégrales entre les plateformes, sans action du développeur
- ☐ Une compatibilité et une abstraction presque intégrales entre les plateformes, qui nécessitent quelques spécificités à gérer côté développeur
- ☐ Une compatibilité partielle, sans aucune abstraction entre les plateformes

Exercice

Avec React Native sur Android, la gestion des ombres portées s'effectue...

- ☐ Grâce à la propriété `elevation` qui prend une valeur numérique
- ☐ En utilisant des propriétés d'ombres telles que `shadowOpacity`
- ☐ Grâce à la propriété `elevation` qui prend une valeur `low` ou `high`

Exercice

1 <https://react-native.canny.io/feature-requests/p/custom-box-shadow-for-android>

2 <https://reactnative.dev/docs/shadow-props>

3 <https://ethercreative.github.io/react-native-shadow-generator/>

Lequel de ces composants ne fonctionne pas sur iOS ?

- ☐ TouchableNativeFeedback
- ☐ TouchableOpacity
- ☐ Button

IV. Écrire du code pour une plateforme spécifique

Objectifs

- Voir les outils pour écrire du code spécifiquement pour une plateforme
- Comprendre quand utiliser la structure conditionnelle et le préfixe de fichiers pour variabiliser son code par plateforme

Mise en situation

Maintenant que nous comprenons qu'il est parfois nécessaire d'écrire du code spécifique par plateforme, nous allons voir comment il faut procéder pour rendre cela effectif. Nous allons utiliser deux manières de procéder, comme nous le permet React Native, chacune ayant ses avantages et ses inconvénients. Ce cours permet de savoir laquelle choisir selon le contexte.

Il existe deux manières de procéder pour une mise en page conditionnelle en fonction de la plateforme d'exécution du code :

- Une pour les différences majeures entre les plateformes,
- Et une autre pour les petits morceaux de code qui changent à des endroits bien précis (pas plus de quelques instructions).

Le préfixe de fichier pour traiter les changements majeurs

React Native utilise un outil qui permet de concentrer tous nos fichiers JavaScript en un seul et unique fichier qui sera lu par l'interpréteur JavaScript du téléphone, le moteur JavaScript.

Sur React Native, il y a deux outils qui sont en charge d'effectuer des traitements sur notre code :

- **Babel**, un transpiler (parfois appelé « compilateur ») pour JavaScript : <https://babeljs.io/>
- **Metro**, le packager de React Native : <https://facebook.github.io/metro/>

Définition Qu'est-ce qu'un transpiler de code ?

Petite parenthèse : un transpiler de code tel que Babel est un outil qui prend du JavaScript en entrée et applique des transformations dessus.

JavaScript étant constamment en évolution, bon nombre de nouvelles fonctionnalités apparaissent avec les nouvelles spécifications. La plupart de ces nouvelles spécifications ne sont en fait que du « sucre syntaxique » (des façons plus simples d'écrire quelque chose de déjà possible), qu'on peut rendre compatible même avec des moteurs JavaScript qui ne le supportent pas encore nativement. C'est très largement utilisé sur le Web, notamment pour la compatibilité navigateur, mais ça l'est également avec React Native.

D'ailleurs, l'équipe React Native maintient une page de documentation dédiée à la transpilation¹.

¹ <https://reactnative.dev/docs/javascript-environment.html>

Définition Qu'est-ce qu'un packager de code ?

Un packager de code est un outil responsable de gérer les imports et la résolution de modules au sein d'une base de code. De cette manière, le packager construit un seul fichier JavaScript, que l'on pourra distribuer et qui contiendra les dépendances du code natif.

Grâce à ces deux outils, il est possible d'appliquer des changements à la volée au moment où les fichiers importent des dépendances depuis d'autres fichiers. Par exemple, quand l'on écrit `import A from "b"`, il est possible d'utiliser des métadonnées du fichier `b` pour venir altérer son contenu ou même complètement l'import lui-même.

Avec tout ça, React Native permet d'utiliser des extensions de fichiers pour définir quelle plateforme utiliser dans un contexte particulier.

Exemple

Nous avons un fichier `MyButton.js` qui contient le code spécifique d'un bouton : or, nous voulons afficher ce bouton en vert sur Android et en bleu sur iOS.

Une des manières de procéder peut être de créer un fichier `MyButton.android.js` pour Android ou `MyButton.ios.js` avec le code spécifique à iOS. Par défaut, si le packager ne trouve pas de fichier spécifique à la plateforme cible, il se chargera de prendre le fichier le plus générique, à savoir `MyButton.js`.

On peut d'ores et déjà essayer le code suivant :

```
1 // App.js
2 import * as React from 'react';
3 import MyButton from './components/MyButton';
4
5 export default function App() {
6   return <MyButton />;
7 }
8
9 // components/MyButton.js
10 import * as React from 'react';
11 import { Button } from 'react-native';
12
13 export default function MyButton() {
14   return (
15     <Button
16       onPress={() => undefined}
17       title="Je suis bleu (iOS / autres)"
18       color="blue"
19     />
20   );
21 }
22
23 // components/MyButton.android.js
24 import * as React from 'react';
25 import { Button } from 'react-native';
26
27 export default function MyButton() {
28   return (
29     <Button
30       onPress={() => undefined}
31       title="Je suis vert (Android)"
32       color="green"
33     />
34   );
35 }
```

```
35 }
```

Cette pratique de séparation en plusieurs fichiers est très simple à utiliser, mais nécessite généralement une partie de duplication de code de par sa nature. Elle offre par contre une entière liberté de programmation.

Conseil

Dans le cas de grosses différences, il est préférable d'utiliser cette manière afin de bien faire la séparation des plateformes et de faciliter la maintenance.

Sous le capot, le packager va choisir au moment de l'assemblage du JavaScript les fichiers correspondant à la plateforme cible du build natif. De cette manière, on embarque uniquement le code responsable de la plateforme cible, ce qui peut alléger un peu le poids de l'application si on a beaucoup de fichiers, même si cela reste négligeable.

Remarque

À noter que cette syntaxe s'applique à tous types de fichiers traités par le packager, tels que les fichiers image ou vidéo. Là, en revanche, ça peut devenir intéressant en termes de poids d'application si nous avons par exemple une grosse vidéo d'introduction dans notre application.

La structure conditionnelle pour les changements limités

La deuxième méthode permet de faire une différenciation conditionnelle via une structure algorithmique au moment où l'application tourne (*runtime*).

Exemple

```
1 import { Platform, StyleSheet } from 'react-native';
2
3 const styles = StyleSheet.create({
4   height: Platform.OS === 'ios' ? 200 : 100
5 });
```

Ici, nous avons donc une hauteur de 200 px sur iOS et une hauteur de 100 px sur Android.

Le module `Platform` permet de détecter la plateforme sur laquelle l'application s'exécute. Il propose une fonction `select()` qui, en fonction de la plateforme actuelle, va retourner la valeur lui correspondant. Cette dernière attend un objet dont les propriétés correspondent aux différentes plateformes disponibles.

Exemple

```
1 import { Platform, StyleSheet } from 'react-native';
2
3 const styles = StyleSheet.create({
4   container: {
5     flex: 1,
6     // On décompose pour assigner la valeur de backgroundColor à l'objet container, de cette
7     // manière il récupère ses propriétés.
8     ...Platform.select({
9       ios: {
10        // Sur iOS, on retourne un objet avec une clé backgroundColor rouge
11        backgroundColor: 'red',
12      },
13      android: {
```

```

13      // Sur android, on retourne un objet avec une clé backgroundColor verte
14      backgroundColor: 'green',
15    },
16    default: {
17      // Par défaut on retourne un objet avec une clé backgroundColor bleu
18      backgroundColor: 'blue',
19    },
20  }),
21 },
22 });

```

Exemple Il est également possible d'accéder à la version du système d'exploitation

```

1 import { Platform } from 'react-native';
2
3 // Sur Android
4 if (Platform.Version === 25) {
5   console.log('Running on Nougat!');
6 }
7
8 // Sur iOS
9 const majorVersionIOS = parseInt(Platform.Version, 10);
10 if (majorVersionIOS <= 9) {
11   console.log('Work around a change in behavior');
12 }

```

Syntaxe À retenir

- Il existe globalement deux manières de procéder à une différenciation des plateformes :
 - Soit par une structure conditionnelle en évaluant la plateforme au *runtime*. Cette manière est simple et légère, et reste préférable quand il y a peu de changements.
 - Soit en préfixant les fichiers, ce qui aura pour effet de choisir le bon fichier à l'assemblage du JavaScript par le packager. Ici, on a une très grande liberté d'algorithmie, par contre cela induit toujours quelques duplications de code. À noter que cette technique s'applique également aux fichiers image et vidéo, ce qui est une très bonne chose !

Complément

- <https://reactnative.dev/docs/platform-specific-code>

V. Exercice : Appliquez la notion

Question

[solution n°2 p.13]

Réalisez une implémentation d'un texte avec un fond rouge (#cf1b1b) sur Android et bleu (#145374) sur iOS en utilisant les deux méthodes décrites dans le cours.

Pour cela, créez un composant `PlatformSpecificText` qui prend en `children` du texte à afficher.

VI. Essentiel

Comme nous réalisons des applications *cross-platform* pour plusieurs supports tel que Android, iOS ou encore le Web, il existe des cas où certaines API et/ou composants ne sont pas supportés sur une de ces plateformes.

React Native permet une grande flexibilité, car l'on peut adapter son code en fonction de la plateforme sur laquelle le code va s'exécuter.

Pour cela, deux manières sont possibles :

- La première est basée sur un préfixe de fichier traité lors de l'empaquetage de l'application.
- La seconde permet de créer des structures conditionnelles qui s'exécutent au *runtime* de l'application et de différencier les systèmes d'exploitation, mais également leur version.

Il incombe au développeur de bien estimer la quantité de code différent qu'il a dans son projet et de choisir la bonne méthode en fonction de cela. Ceci impactera notamment la maintenance de l'application, mais pourrait également impacter son poids en termes de mega-octets, notamment sur un paquetage conditionnel de fichiers vidéos/photos.

VII. Auto-évaluation

A. Exercice final

Exercice 1

[solution n°3 p.15]

Exercice

Quel est le nom du packager de React Native ?

- ☐ ReactNativePackager
- ☐ MetroBundler
- ☐ TrainBundler
- ☐ Yoga

Exercice

Quel est le transpiler utilisé par React Native ?

- ☐ Webpack
- ☐ RollupJS
- ☐ BabelJS
- ☐ JsTranspilerX

Exercice

On pourrait dire qu'un transpiler c'est...

- ☐ Un outil qui corrige les fautes d'orthographe dans le code
- ☐ Un outil qui vérifie l'intégrité et la cohérence du code
- ☐ Un outil qui prend du code en entrée et y applique des modifications si nécessaire

Exercice

On pourrait dire qu'un packager, c'est...

- ☐ Un outil qui prend du code en entrée, résout les dépendances et génère un ou des fichiers finaux optimisés
- ☐ Un outil qui corrige les fautes d'orthographe dans le code
- ☐ Un outil qui vérifie l'intégrité et la cohérence du code : il est par exemple capable de corriger le manque d'un guillemet

Exercice

Que fait iOS si on lui passe une propriété `elevation` ?

- ☐ Il va calculer automatiquement l'ombre portée adéquate pour iOS
- ☐ Il l'ignore complètement sans planter
- ☐ L'application plante

Exercice

Comment s'appelle l'API qui permet d'évaluer sur quelle plateforme on se trouve actuellement ?

- ☐ Platform
- ☐ PlatformEnvironment
- ☐ Environment

Exercice

Ayant deux fichiers vidéo de 15 mega-octets pour afficher respectivement sur Android et iOS, qu'est-il pertinent de faire ?

- ☐ Utiliser l'API `Platform` au *runtime*
- ☐ Utiliser les préfixes de fichier par rapport aux plateformes

Exercice

Android tire sa notion d'« elevation » de...

- ☐ La spécification Human Interfaces d'Apple
- ☐ La spécification Material Design de Google
- ☐ La spécification Android MobileDesign de Google

Exercice

Pour récupérer la version du système d'exploitation, on utilise...

- ☐ `Platform.version`
- ☐ `Platform.OSVersion`
- ☐ `Platform.getVersion()`, une fonction

Exercice

En suivant le lien vers la documentation sur « JavaScript Environment » présent dans cet article, quel élément est fourni en support par le transpiler ?

- ☐ ArrowFunction
- ☐ Array.length
- ☐ Platform.version

B. Exercice : Défi

Très bien ! Maintenant que nous connaissons le rendu conditionnel par plateforme, nous allons jouer avec les ombres portées sur les éléments et tenter de faire correspondre ces dernières entre les plateformes, malgré les différences sous-jacentes.

Question

[solution n°4 p.17]

Créons maintenant un composant `PlatformSpecificCard` qui affiche une simple boîte de largeur 100 par 100, avec un arrière-plan rouge (code couleur `#cf1b1b`) sur Android et bleu (`#145374`) sur iOS.

Cette boîte devra afficher une ombre portée pour donner un effet de hauteur dans l'interface, en utilisant la propriété vue dans ce cours.

La puissance de l'ombre sera passée en paramètre via une unique propriété `elevation`, certes supportée par Android, mais pas par iOS.

Il faudra déduire, grâce à la valeur de l'élévation, les valeurs des propriétés `shadowOpacity`, `shadowRadius`, et `shadowOffset` pour iOS en faisant quelques recherches sur Internet sur des algorithmes qui s'en rapprochent.

Ensuite, il faudra afficher deux composants `PlatformSpecificCard` : un avec une élévation de 0, et l'autre de 8.

En cas de blocage sur la recherche de cet algorithme d'équivalence sur le Web, un algorithme qui se rapproche du rendu escompté est donné en indice ci-dessous.

Indice :

L'algorithme d'approximation d'une ombre à partir d'une élévation pour iOS :

```
1 const iosShadowElevation = {  
2   shadowOpacity: 0.0015 * elevation + 0.18,  
3   shadowRadius: 0.54 * elevation,  
4   shadowOffset: {  
5     height: 0.6 * elevation,  
6   },  
7 };
```

Solutions des exercices

Exercice p. 4 Solution n°1**Exercice**

React Native assure...

- ☐ Une compatibilité et une abstraction intégrales entre les plateformes, sans action du développeur
- ☒ Une compatibilité et une abstraction presque intégrales entre les plateformes, qui nécessitent quelques spécificités à gérer côté développeur
- ☐ Une compatibilité partielle, sans aucune abstraction entre les plateformes
-  React Native fait au mieux pour fournir une abstraction pour rendre le code compatible sur les plateformes, en utilisant des composants les plus proches possible. Cependant, ce n'est pas toujours le cas et, dans certains cas, le développeur devra intervenir pour faire varier son code en fonction du contexte.

Exercice

Avec React Native sur Android, la gestion des ombres portées s'effectue...

- ☒ Grâce à la propriété `elevation` qui prend une valeur numérique
- ☐ En utilisant des propriétés d'ombres telles que `shadowOpacity`
- ☐ Grâce à la propriété `elevation` qui prend une valeur `low` ou `high`
-  Sur Android, les ombres sont représentées par une propriété `elevation` qui prend un nombre indiquant le degré de cette élévation.

Exercice

Lequel de ces composants ne fonctionne pas sur iOS ?

- ☒ `TouchableNativeFeedback`
- ☐ `TouchableOpacity`
- ☐ `Button`
-  `TouchableOpacity` et `Button` sont deux composants permettant de gérer des zones cliquables et sont compatibles sur iOS et Android. D'ailleurs, `Button` est un composant de haut niveau qui utilise `TouchableOpacity` sur iOS et `TouchableNativeFeedback` sur Android, car ce dernier n'est utilisable que sur Android.

p. 8 Solution n°2**Première méthode avec l'API Platform**

```

1 // App.js
2 import * as React from 'react';
3 import { Text, StyleSheet, Platform } from 'react-native';
4
5 export default function App() {
6   return <PlatformSpecificText>Hello</PlatformSpecificText>;
7 }
8
9 function PlatformSpecificText(props) {
10   return <Text style={platformTextStyles.text}>{props.children}</Text>;
11 }

```

```

12
13 const platformTextStyles = StyleSheet.create({
14   text: {
15     fontSize: 24,
16     textAlign: 'center',
17     // On utilise l'API Platform
18     backgroundColor: Platform.OS === 'android' ? '#cf1b1b' : '#145374',
19     color: 'white',
20     padding: 5,
21   },
22 });

```

Seconde méthode avec le préfixe de fichiers

Contenu du fichier App.js :

```

1 import * as React from 'react';
2 import { PlatformSpecificText } from './components/PlatformSpecificText'
3
4 export default function App() {
5   return <PlatformSpecificText>Hello</PlatformSpecificText>;
6 }

```

Contenu d'un fichier components/PlatformSpecificText.android.js :

```

1 import React from 'react';
2 import { Text, StyleSheet } from 'react-native';
3
4 export function PlatformSpecificText(props) {
5   return <Text style={platformTextStyles.text}>{props.children}</Text>;
6 }
7
8 const platformTextStyles = StyleSheet.create({
9   text: {
10     fontSize: 24,
11     textAlign: 'center',
12     backgroundColor: '#cf1b1b', // La bonne couleur
13     color: 'white',
14     padding: 5,
15   },
16 });

```

Contenu d'un fichier components/PlatformSpecificText.js :

```

1 import React from 'react';
2 import { Text, StyleSheet } from 'react-native';
3
4 export function PlatformSpecificText(props) {
5   return <Text style={platformTextStyles.text}>{props.children}</Text>;
6 }
7
8 const platformTextStyles = StyleSheet.create({
9   text: {
10     fontSize: 24,
11     textAlign: 'center',
12     backgroundColor: '#145374', // La bonne couleur
13     color: 'white',
14     padding: 5,
15   },
16 });

```

Exercice p. 9 Solution n°3**Exercice**

Quel est le nom du packager de React Native ?

- ☐ ReactNativePackager
- ☒ MetroBundler
- ☐ TrainBundler
- ☐ Yoga
-  Yoga est le moteur de rendu Flexbox et les autres n'existent pas ou sont une fonction.

Exercice

Quel est le transpiler utilisé par React Native ?

- ☐ Webpack
- ☐ RollupJS
- ☒ BabelJS
- ☐ JsTranspilerX
-  La bonne réponse est BabelJS, le transpiler le plus populaire dans l'univers JavaScript.

Exercice

On pourrait dire qu'un transpiler c'est...

- ☐ Un outil qui corrige les fautes d'orthographe dans le code
- ☐ Un outil qui vérifie l'intégrité et la cohérence du code
- ☒ Un outil qui prend du code en entrée et y applique des modifications si nécessaire
-  Un transpiler permet principalement d'utiliser des fonctionnalités type « sucre syntaxique », disponibles uniquement sur des versions très récentes. Il applique donc les transformations nécessaires pour les rendre rétrocompatibles.

Exercice

On pourrait dire qu'un packager, c'est...

- ☒ Un outil qui prend du code en entrée, résout les dépendances et génère un ou des fichiers finaux optimisés
- ☐ Un outil qui corrige les fautes d'orthographe dans le code
- ☐ Un outil qui vérifie l'intégrité et la cohérence du code : il est par exemple capable de corriger le manque d'un guillemet
-  Un packager, comme son nom l'indique, génère et gère des paquets applicatifs. Il permet donc de préparer le package d'une application React Native.

Exercice

Que fait iOS si on lui passe une propriété `elevation` ?

- ☐ Il va calculer automatiquement l'ombre portée adéquate pour iOS
- ☒ Il l'ignore complètement sans planter
- ☐ L'application plante
-  En effet, il existe des sécurités pour que ça ne plante pas : cependant, nous n'aurons pas le résultat escompté. Pour ce qui est de calculer automatiquement, nous allons le faire dans le prochain exercice « Défi ».

Exercice

Comment s'appelle l'API qui permet d'évaluer sur quelle plateforme on se trouve actuellement ?

- ☒ Platform
- ☐ PlatformEnvironment
- ☐ Environment
-  Seule `Platform` existe réellement.

Exercice

Ayant deux fichiers vidéo de 15 mega-octets pour afficher respectivement sur Android et iOS, qu'est-il pertinent de faire ?

- ☐ Utiliser l'API `Platform` au *runtime*
- ☒ Utiliser les préfixes de fichier par rapport aux plateformes
-  En utilisant les préfixes, les fichiers ne seront pas importés s'ils ne sont pas utilisés dans l'application : on gagne alors plusieurs mega-octets d'espace pour notre application à télécharger.

Exercice

Android tire sa notion d'« elevation » de...

- ☐ La spécification Human Interfaces d'Apple
- ☒ La spécification Material Design de Google
- ☐ La spécification Android MobileDesign de Google
-  La bonne réponse est Material Design : <https://material.io/design/environment/elevation.html>.

Exercice

Pour récupérer la version du système d'exploitation, on utilise...

- ☒ `Platform.version`
- ☐ `Platform.OSVersion`
- ☐ `Platform.getVersion()`, une fonction
-  La bonne réponse est `Platform.version`, une constante accessible au *runtime*.

Exercice

En suivant le lien vers la documentation sur « JavaScript Environment » présent dans cet article, quel élément est fourni en support par le transpiler ?

☒ ArrowFunction

☐ Array.length

☐ Platform.version

 La bonne réponse est ArrowFunction : <https://reactnative.dev/docs/javascript-environment.html#javascript-syntax-transformers>.

p. 11 Solution n°4

```
1 // App.js
2 import * as React from 'react';
3 import { View, StyleSheet, Platform } from 'react-native';
4
5 export default function App() {
6   return (
7     <View style={platformCardStyles.container}>
8       <PlatformSpecificCard elevation={0} />
9       <PlatformSpecificCard elevation={8} />
10    </View>
11  );
12 }
13
14 export function PlatformSpecificCard(props) {
15   // Dans le cas où il n'y a pas d'elevation
16   if (typeof props.elevation === 'undefined' || props.elevation === 0) {
17     return <View style={platformCardStyles.card} />;
18   }
19
20   // Cas android
21   if (Platform.OS === 'android') {
22     return (
23       <View
24         elevation={props.elevation}
25         style={[{ elevation: props.elevation }, platformCardStyles.card]}
26       />
27     );
28   }
29
30   // Cas iOS, selon algorithme d'approximation
31   return (
32     <View
33       style={[
34         {
35           shadowOpacity: 0.0015 * props.elevation + 0.18,
36           shadowRadius: 0.54 * props.elevation,
37           shadowOffset: {
38             height: 0.6 * props.elevation,
39           },
40         },
41         platformCardStyles.card,
42       ]
43     >
```

```
43     />
44   );
45 }
46
47 const platformCardStyles = StyleSheet.create({
48   container: {
49     flex: 1,
50     justifyContent: 'space-around',
51     alignItems: 'center',
52   },
53   card: {
54     backgroundColor: Platform.OS === 'android' ? '#cf1b1b' : '#145374',
55     height: 100,
56     width: 100,
57   },
58 });
```